

LLDD BE - Job Batch and Email Integration

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจสอบ และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. Overview

รายการ	รายละเอียด
Track	BE
Estimate	15 ชั่วโมง
Owner	Peerakorn <Pete> Sakunkaewphithak
Objective	ออกแบบ Backend contracts สำหรับ batch runner (อ่าน config จาก backend), interface tracking/pending ACK และ Notification Service (ส่งผ่าน @gosoft-sbp/email-lib) — ไม่มี Job Admin API, Email Template API (2026-08-06) และไม่มี SRM inbound adapter แล้ว (2026-08-07)

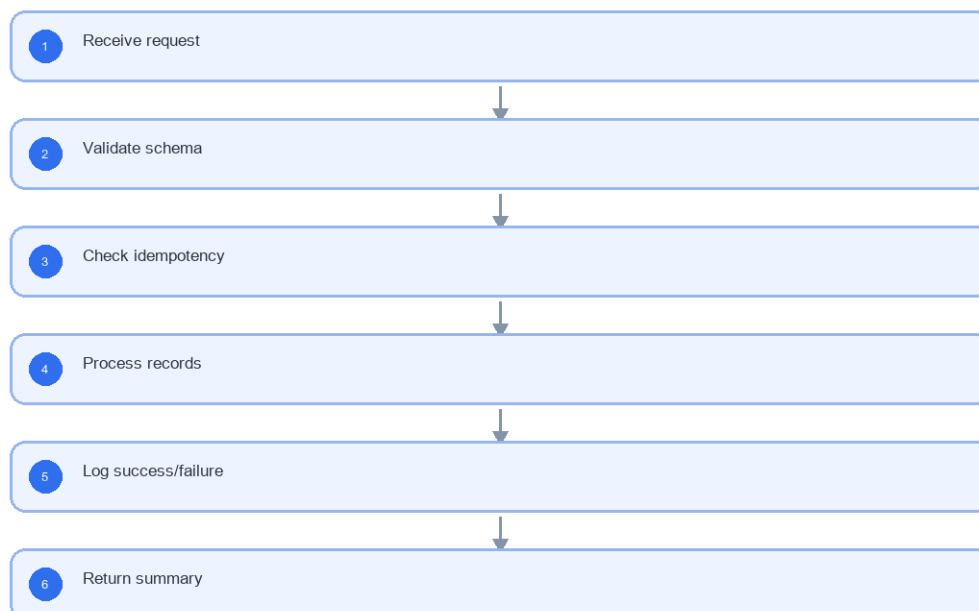
Common contract reference: ทุกหัวข้อ API/FE ต้องยึด LLDD-BE-API-Common-Contracts.pdf และ LLDD-FE-Integration-Contracts.pdf สำหรับ error/auth/format/pagination/action/RBAC ก่อนลงรายละเอียดเฉพาะหน้าหรือเฉพาะ endpoint

2. Screen / Functional Scope

- Interface tracking และ pending ACK APIs (3 เส้น)
- Job runner guard และ application log
- Notification adapter ผ่าน @gosoft-sbp/email-lib
- STA ACK callback
- ไม่มี Batch Job Admin API และไม่มี inbound endpoint ของ SRM

4. Implementation Flow Diagram (Reference)

LLDD BE - Job Batch and Email Integration



รูปที่ 1: Implementation flow reference: LLDD BE - Job Batch and Email Integration

5. Field, Format, and Validation

Field / UI	Format	Validation	Behavior
jobNo	string	required	maps to job registry
sourceRefNo	string	required for SRM	idempotency key
templateCode	EM-xx	required	email template key
transactionId	uuid	generated	integration log key

5.1 Input / Progress / Output Contract

Stage	Contract for implementation
Input	GET /api/v1/interfaces/tracking; GET /api/v1/interfaces/pending-ack; POST /api/v1/interfaces/sta/ack
Progress	Receive request; Validate schema; Check idempotency; Process records
Output	job_configs; job_run_histories; interface_transactions

5.90 Endpoint Implementation Contract

Endpoint	Use-case owner	Service/repository behavior	Definition of done
GET /api/v1/interfaces/tracking	ค้นสถานะ interface ตาม dataset/business key/status/ช่วงเวลา	Receive request	job run guard prevents duplicate running job
GET /api/v1/interfaces/pending-ack	รายการ ACK ค้างตาม watchdog rule อย่างน้อย 1 วัน	Validate schema	email preview renders variables
POST /api/v1/interfaces/sta/ack	STA ACK callback ให้ Job 10 เป็น safety net	Check idempotency	failed records include detail

5.91 Backend Execution Sequence

Step	Behavior specific to this LLDD	Failure/test evidence
1	Receive request	run job
2	Validate schema	run duplicate
3	Check idempotency	interface tracking filter
4	Process records	pending ACK watchdog
5	Log success/failure	STA ACK callback
6	Return summary	email preview

6. Button / User Action Mapping

Action	Trigger	API / Service	Expected Result
Run job	POST	jobRunner.run	queued/run history
Receive SRM	POST	srmlIntegration.ingest	transaction result
Preview email	POST	emailTemplate.render	merged subject/body

7. API Contract

GET /api/v1/interfaces/tracking

ค้นหาสถานะ interface ตาม dataset/business key/status/ช่วงเวลา

Query Params

```
{
  "dataName": "COMPENSATE_INIT_I",
  "status": "SENT",
  "pending": true,
  "sentFrom": "2026-07-01T00:00:00+07:00",
  "sentTo": "2026-07-22T23:59:59+07:00",
  "page": 1,
  "size": 20
}
```

Request Field Schema

Field	Type	Required	Constraint / Meaning
dataName	string	No	UTF-8; use value domain described by endpoint purpose
status	string	No	UTF-8; use value domain described by endpoint purpose
pending	boolean	No	UTF-8; use value domain described by endpoint purpose
sentFrom	string	No	UTF-8; use value domain described by endpoint purpose
sentTo	string	No	UTF-8; use value domain described by endpoint purpose
page	integer	No	>= 1; default 1
size	integer	No	1..100; default 20

Response

```
{
  "page": 1,
  "size": 20,
  "total": 1,
  "items": [
    {
      "trackingId": 9912,
      "dataName": "COMPENSATE_INIT_I",
      "direction": "OUT",
      "businessKey": "2026/00098",
      "docNo": "2026/00098",
      "fileName": "COMPENSATE_INIT_I_25690722.dat",
      "status": "SENT",
      "sentAt": "2026-07-20T17:02:00+07:00",
      "ackedAt": null,
      "returnCode": null,
      "ageHours": 41
    }
  ]
}
```

Response Field Schema

Field	Type	Required	Constraint / Meaning
page	integer	Yes	>= 1; default 1
size	integer	Yes	1..100; default 20
total	integer	Yes	UTF-8; use value domain described by endpoint purpose

Field	Type	Required	Constraint / Meaning
items	array<object>	Yes	JSON array; element type shown in Type column
items[].trackingId	integer	Yes	UTF-8; use value domain described by endpoint purpose
items[].dataName	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].direction	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].businessKey	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].docNo	string	Yes	พ.ศ. YYYY/xxxxx
items[].fileName	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].status	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].sentAt	string	Yes	ISO-8601 ค.ศ.; nullable only when type includes null
items[].ackedAt	string null	No	ISO-8601 ค.ศ.; nullable only when type includes null
items[].returnCode	string null	No	UTF-8; use value domain described by endpoint purpose
items[].ageHours	integer	Yes	UTF-8; use value domain described by endpoint purpose

GET /api/v1/interfaces/pending-ack

รายการ ACK ค้างตาม watchdog rule อายุอย่างน้อย 1 วัน

Query Params
<pre>{ "thresholdHours": 24, "dataName": "COMPENSATE_INIT_I", "page": 1, "size": 20 }</pre>

Request Field Schema

Field	Type	Required	Constraint / Meaning
thresholdHours	integer	No	UTF-8; use value domain described by endpoint purpose
dataName	string	No	UTF-8; use value domain described by endpoint purpose
page	integer	No	>= 1; default 1
size	integer	No	1..100; default 20

Response

```
{
  "page": 1,
  "size": 20,
  "total": 1,
  "count": 1,
  "items": [
    {
      "trackingId": 9912,
      "dataName": "COMPENSATE_INIT_I",
      "businessKey": "2026/00098",
      "docNo": "2026/00098",
      "fileName": "COMPENSATE_INIT_I_25690722.dat",
      "sentAt": "2026-07-20T17:02:00+07:00",
      "ageHours": 41,
      "returnCode": null
    }
  ]
}
```

Response Field Schema

Field	Type	Required	Constraint / Meaning
page	integer	Yes	>= 1; default 1
size	integer	Yes	1..100; default 20
total	integer	Yes	UTF-8; use value domain described by endpoint purpose
count	integer	Yes	UTF-8; use value domain described by endpoint purpose
items	array<object>	Yes	JSON array; element type shown in Type column
items[].trackingId	integer	Yes	UTF-8; use value domain described by endpoint purpose
items[].dataName	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].businessKey	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].docNo	string	Yes	พ.ศ. YYYY/xxxxx
items[].fileName	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].sentAt	string	Yes	ISO-8601 ค.ศ.; nullable only when type includes null
items[].ageHours	integer	Yes	UTF-8; use value domain described by endpoint purpose
items[].returnCode	string null	No	UTF-8; use value domain described by endpoint purpose

POST /api/v1/interfaces/sta/ack

STA ACK callback ให้ Job 10 เป็น safety net

Request

```
{
  "transactionId": "TX-001",
  "returnCode": "A",
  "receivedAt": "2026-07-20T10:00:00+07:00"
}
```

Request Field Schema

Field	Type	Required	Constraint / Meaning
transactionId	string	Yes	UTF-8; use value domain described by endpoint purpose
returnCode	string	Yes	UTF-8; use value domain described by endpoint purpose
receivedAt	string	Yes	ISO-8601 ค.ศ.; nullable only when type includes null

Response

```
{
  "message": "acknowledged"
}
```

Response Field Schema

Field	Type	Required	Constraint / Meaning
message	string	Yes	UTF-8; use value domain described by endpoint purpose

8. Reference DB Mapping (No Database Page Work)

ส่วนนี้เป็นข้อมูลอ้างอิงสำหรับการ implement API/Job เท่านั้น ไม่ใช้งานสร้างหน้า Database, ไม่ใช้งานออกแบบ DB page และไม่ถูกนับเป็น deliverable แยกของ FE/BE

Table / Object	R/W	Usage
job_configs	R/W	enabled, cron, params ของ batch
job_run_histories	R/W	ประวัติการรันและสถานะล่าสุด
interface_transactions	R/W	tracking file/API interface และ ACK
email_template (SBP)	R/W	subject_format/body_format ของระบบ SBP เดิม
status_email_rules	R	TO/CC ตามสถานะ

9. Skeleton Code (store-backend + BFF)

โครงโค้ดตั้งต้นของเอกสารฉบับนี้ ยึด convention จริงของ srm-sps-spsap-store-backend (NestJS 11 + TypeORM, schema sps_store, custom provider DATA_SOURCE ที่ route SELECT ไป slave pool) และ srm-sps-spsap-sbp-bff (ไม่มี DB, forward ผ่าน client service). ทุกจุดที่ต้องเติมกำกับด้วย // TODO: และ response ทุกเส้นถูกห่อเป็น {success, data} โดย ResponseInterceptor อยู่แล้ว จึงห้าม service ห่อซ้ำ

9.1 ไฟล์ที่ต้องสร้าง

Path	หน้าที่
store-backend · src/modules/sbpqi-job-batch-email-srm/sbpqi-job-batch-email-srm.controller.ts	route ทั้งหมดของเอกสารนี้ (3 เส้น) + @UseGuards(HttpHeaderGuard) + @UserId()
store-backend · src/modules/sbpqi-job-batch-email-srm/sbpqi-job-batch-email-srm.service.ts	business logic — inject 'DATA_SOURCE' แล้วยิง raw SQL, mutation ใช้ QueryRunner transaction
store-backend · src/modules/sbpqi-job-batch-email-srm/sbpqi-job-batch-email-srm.sql.ts	เก็บ SQL ต่อ endpoint (คัดจากหัวข้อ 10) แยกออกจาก service ให้ทดสอบ/รีวิวง่าย
store-backend · src/modules/sbpqi-job-batch-email-srm/dto/sbpqi-job-batch-email-srm.dto.ts	DTO + class-validator ตาม validation ในหัวข้อฟิลด์ของเอกสารนี้
store-backend · src/modules/sbpqi-job-batch-email-srm/sbpqi-job-batch-email-srm.module.ts	ประกอบ controller/service/providers แล้ว register ที่ app.module.ts

Path	หน้าที่
store-backend · src/entities/job-configs.entity.ts	entity ของ job_configs (@Entity({schema: process.env.DB_SCHEMA})), ไม่ประกาศ relation)
store-backend · src/entities/job-run-histories.entity.ts	entity ของ job_run_histories (@Entity({schema: process.env.DB_SCHEMA})), ไม่ประกาศ relation)
store-backend · src/entities/interface-transactions.entity.ts	entity ของ interface_transactions (@Entity({schema: process.env.DB_SCHEMA})), ไม่ประกาศ relation)
store-backend · src/providers/sbpgi/sbpgi.ts	repository provider แบบ factory ผูก token string กับ DATA_SOURCE — ไฟล์รวมของทุกเอกสาร BE ให้ merge array เพิ่ม ห้ามเขียนทับ
store-backend · sql/deploy-sbpgi-job-batch-email-srm.sql	DDL production แบบ idempotent (ทีมนี้ไม่ใช่ migration เป็นหลัก)
BFF · src/common/client-services/sbpgi-client.service.ts	client ต่อจาก BaseClientService ตั้ง baseUrl + x-api-key ตอน onModuleInit
BFF · src/modules/sbpgi-job-batch-email-srm/sbpgi-job-batch-email-srm.controller.ts	route ผัง BFF prefix /bff/sbpgi/... + @UseGuards(AuthGuard('jwt'))
BFF · src/modules/sbpgi-job-batch-email-srm/sbpgi-job-batch-email-srm.service.ts	แนบ x-user-id / x-user-group-id / x-user-permissions แล้ว forward ไป backend

9.2 Controller (store-backend)

```
// src/modules/sbpgi-job-batch-email-srm/sbpgi-job-batch-email-srm.controller.ts
import { Body, Controller, Get, Post, Query, UseGuards } from '@nestjs/common';
import { HttpHeaderGuard } from '../../guards/http-header.guard';
import { UserId } from '../../common/decorators/user-id.decorator';
import { SbpgiJobBatchEmailSRMService } from './sbpgi-job-batch-email-srm.service';
import { JobBatchEmailSRMQueryDto, ReceiveAckStaBodyDto } from './dto/sbpgi-job-batch-email-srm.dto';

// LLDD BE - Job Batch and Email Integration
// BFF เรียกด้วย x-api-key และแนบ x-user-id / x-user-group-id / x-user-permissions มาให้
@Controller('sbpgi/interfaces')
@UseGuards(HttpHeaderGuard)
export class SbpgiJobBatchEmailSRMController {
  constructor(private readonly service: SbpgiJobBatchEmailSRMService) {}

  // GET /api/v1/interfaces/tracking — ค้นสถานะ interface ตาม dataset/business key/status/ช่วงเวลา
  @Get('tracking')
  getInterfacesTracking(@Query() query: JobBatchEmailSRMQueryDto, @UserId() userId: string) {
    // TODO: ตรวจสอบ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
    return this.service.getInterfacesTracking(query, userId);
  }

  // GET /api/v1/interfaces/pending-ack — รายการ ACK ค้างตาม watchdog rule อายุอย่างน้อย 1 วัน
  @Get('pending-ack')
  getInterfacesPendingAck(@Query() query: JobBatchEmailSRMQueryDto, @UserId() userId: string) {
    // TODO: ตรวจสอบ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
    return this.service.getInterfacesPendingAck(query, userId);
  }

  // POST /api/v1/interfaces/sta/ack — STA ACK callback ให้ Job 10 เป็น safety net
  @Post('sta/ack')
  receiveAckSta(@Body() body: ReceiveAckStaBodyDto, @UserId() userId: string) {
    // TODO: ตรวจสอบ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
    return this.service.receiveAckSta(body, userId);
  }
}
```

9.3 DTO + Validation

```
// src/modules/sbpgi-job-batch-email-srm/dto/sbpgi-job-batch-email-srm.dto.ts
import { Type } from 'class-transformer';
import {
  IsArray, IsBoolean, IsIn, IsInt, IsNotEmpty, IsNumber, IsObject, IsOptional,
  IsString, Matches, Max, MaxLength, Min,
} from 'class-validator';

// ValidationPipe ระดับ global ตั้ง whitelist + forbidNonWhitelisted + transform ไว้แล้ว (main.ts)
// property ที่ไม่ประกาศที่นี่จะถูก reject เป็น 400 อัตราอัตโนมัติ

// query รวมของ GET ทุกเส้นในโมดูลนี้ (path param ใช้ @Param แยก)
export class JobBatchEmailSRMQueryDto {
  @IsNotEmpty()
  @IsString()
  dataName: string;
```



```

/** required เฉพาะบาง endpoint — ตรวจสอบใน service */
@IsOptional()
@IsString()
status?: string;

/** required เฉพาะบาง endpoint — ตรวจสอบใน service */
@IsOptional()
@Type(() => Boolean)
@IsBoolean()
pending?: boolean;

/** required เฉพาะบาง endpoint — ตรวจสอบใน service */
@IsOptional()
@IsString()
sentFrom?: string;

/** required เฉพาะบาง endpoint — ตรวจสอบใน service */
@IsOptional()
@IsString()
sentTo?: string;

@IsOptional()
@Type(() => Number)
@IsInt()
@Min(1)
page?: number;

// TODO: เพิ่ม property ที่เหลือของ payload นี้ให้ครบตามหัวข้อฟิลด์ของเอกสารนี้
,
// body ของ POST /api/v1/interfaces/sta/ack
export class ReceiveAckStaBodyDto {
  /** integration log key */
  @IsNotEmpty()
  @IsString()
  transactionId: string;

  @IsNotEmpty()
  @IsString()
  returnCode: string;

  @IsNotEmpty()
  @IsString()
  receivedAt: string;
}

```

9.4 Service (inject `DATA\_SOURCE` + raw SQL)

service ประกาศ method ครบทุกเส้นที่ controller เรียก และ signature มาจากแหล่งเดียวกับ controller (จำนวน/ลำดับพารามิเตอร์จึงตรงกันเสมอ) — เส้นที่ยังไม่ได้ implement เป็น stub ที่ throw new `NotImplementedException(...)` ให้ TypeScript compile ผ่านตั้งแต่วันแรก

```

// src/modules/sbpgi-job-batch-email-srm/sbpgi-job-batch-email-srm.service.ts
import { Inject, Injectable, Logger, NotFoundException, NotImplementedException } from '@nestjs/common';
import { DataSource } from 'typeorm';
import { WorkflowService } from '../workflow/workflow.service';
import { SBPGI_SQL } from './sbpgi-job-batch-email-srm.sql';

@Injectable()
export class SbpgiJobBatchEmailSRMService {
  private readonly logger = new Logger(SbpgiJobBatchEmailSRMService.name);
  // versionId ของ workflow ประกันรายได้ (ตั้งใน env เหมือน COOPERATION_WORKFLOW_VERSION_ID)
  private readonly versionId = Number(process.env.SBPGI_WORKFLOW_VERSION_ID);

  constructor(
    // DATA_SOURCE override query(): SELECT/WITH ไป slave pool, write ไป master
    @Inject('DATA_SOURCE') private readonly dataSource: DataSource,
    private readonly workflow: WorkflowService,
  ) {}

  // GET /api/v1/interfaces/tracking — ค้นสถานะ interface ตาม dataset/business key/status/ช่วงเวลา
  async getInterfacesTracking(query: JobBatchEmailSRMQueryDto, userId: string) {
    const page = Number(query.page ?? 1);
    const size = Math.min(Number(query.size ?? 20), 100);
    // SQL เติมอยู่ในหัวข้อ Database SQL ของเอกสารนี้ (คือ 'GET /api/v1/interfaces/tracking')
    // □ □ SQL ตัวอย่างบางเส้นเขียนด้วย named parameter (:size/:offset) แต่ dataSource.query()
    // รับเฉพาะ positional $1..$n — ต้องแปลงชื่อเป็นลำดับก่อน หรือใช้ QueryBuilder แทน
    const rows = await this.dataSource.query(SBPGI_SQL.getInterfacesTracking, [
      // TODO: เรียงพารามิเตอร์ให้ตรงกับ $1..$n ของ SQL จริง
      userId, (page - 1) * size, size,
    ]);
    // TODO: total ต้องมาจาก COUNT(*) แยก query หรือ window function ไม่ใช่ rows.length
  }
}

```

```

return { page, size, total: rows.length, items: rows };
}

// GET /api/v1/interfaces/pending-ack — รายการ ACK ค้างตาม watchdog rule อายุอย่างน้อย 1 วัน
async getInterfacesPendingAck(query: JobBatchEmailSRMQueryDto, userId: string) {
  // TODO: implement ตาม business rule ของ GET /api/v1/interfaces/pending-ack
  // (SQL อยู่ในหัวข้อ Database SQL คีย์ 'GET /api/v1/interfaces/pending-ack')
  throw new NotImplementedException('getInterfacesPendingAck ยังไม่ implement');
}

// POST /api/v1/interfaces/sta/ack — STA ACK callback ให้ Job 10 เป็น safety net
// mutation ต้องอยู่ใน transaction เดียว (ไม่มี audit ของ master แล้ว · 2026-08-07)
async receiveAckSta(body: ReceiveAckStaBodyDto, userId: string) {
  const runner = this.dataSource.createQueryRunner();
  await runner.connect();
  await runner.startTransaction();
  try {
    // TODO: lock แถวเป้าหมายของ job_configs ด้วย SELECT ... FOR UPDATE ก่อนเขียน
    const [current] = await runner.query(SBPGI_SQL.receiveAckStaLock, [body.docNo]);
    if (!current) {
      throw new NotFoundException("ไม่พบข้อมูลที่ต้องการ");
    }
    await runner.query(SBPGI_SQL.receiveAckSta, [/* TODO: ผูกค่าจาก body */]);
    await runner.commitTransaction();
    // □□ workflow engine อยู่คนละ DataSource ('workflow-connection' ของ @srm/glb-workflow)
    // จึง **atomic รวมกับ transaction ข้างบนไม่ได้** — ต้อง commit ผัง SBPGI ให้เสร็จก่อน
    // แล้วค่อย triggerEvent (idempotency key = referenceld = docNo)
    // TODO: เรียก workflow use case ตามตารางหัวข้อ Workflow ด้านล่าง + retry
    // TODO: ถ้า triggerEvent ล้มเหลว ต้องมี compensating action และบันทึกผลลง
    // consideration_logs เพื่อให้ job reconcile ตามเก็บได้
    return { message: 'saved' };
  } catch (error) {
    await runner.rollbackTransaction();
    this.logger.error(error);
    throw error;
  } finally {
    await runner.release();
  }
}
}
}

```

9.5 Workflow (`@srm/glb-workflow`)

□□ ชื่อ function ของ engine ยังไม่ยืนยัน (บันทึก 2026-08-07) — แหล่งอ้างอิง 3 แหล่งให้ชื่อไม่ตรงกัน ชุด A TSM SRM LLDD SBP workflow ■■■■ 1.2 ชุด Detail = eventWorkflow · addPreApprover · getPendingFlowByUser · ชุด B ชุด Mermaid seq ของไฟล์เดียวกัน = triggerEvent · ชุด C srm sps spsap store backend §1.5 = TriggerEventUseCase · AddPreparedApproverUseCase · GetPendingFlowUseCase · ชื่อที่ใช้ใน skeleton ด้านล่างเป็น ชื่อชั่วคราว ต้องยืนยันกับทีมเจ้าของ library ก่อนเขียนโค้ดจริง (ดู LLDD-BE-Workflow-Engine-Definition.pdf หัวข้อ 5.3) · engine มี 13 ตาราง อยู่ใน schema sps_store (ไม่ใช่ 10 ตาราง และไม่ใช่ sps_auth)

Endpoint	Use case ที่ต้องเรียก	เหตุผล
GET /api/v1/interfaces/pending-ack	getPendingFlow()	inbox งานค้างของ userId/groupId ที่ BFF ส่งมาใน header

```

// src/modules/sbpqi-job-batch-email-srm/sbpqi-job-batch-email-srm.workflow.ts (หรือรวมไว้ใน service เดียวกัน)
// WorkflowService = wrapper ของ @srm/glb-workflow ที่ store-backend มีอยู่แล้ว
// (DataSource แยกชื่อ 'workflow-connection', ทุก use case ห่อด้วย TypeOrmUnitOfWork)

// inbox งานค้าง — ใช้ร่วมกับ /api/workflow/pending ของ backlog เดิมได้
const pending = await this.workflow.getPendingFlow({
  userData: { userId: Number(userId), groupId: Number(groupId) },
  versionId: this.versionId,
});
// TODO: join referenceld (= doc_no) กลับไปที่ compensation_documents เพื่อเติมข้อมูลเอกสาร

```

9.6 Entity (TypeORM)

```

// src/entities/job-configs.entity.ts
import { Column, Entity, PrimaryColumn } from 'typeorm';

@Entity({ name: 'job_configs', schema: process.env.DB_SCHEMA })
export class JobConfig {
  @PrimaryColumn({ name: 'job_no', type: 'varchar', length: 5 })
  jobNo: string;

  @Column({ name: 'job_name', type: 'varchar', length: 200 })
  jobName: string;
}

```

```

@Column({ name: 'cron_expression', type: 'varchar', length: 50, nullable: true })
cronExpression?: string;

@Column({ name: 'is_enabled', type: 'boolean', default: true })
isEnabled: boolean;

@Column({ name: 'params', type: 'jsonb', nullable: true })
params?: Record<string, unknown>;

@Column({ name: 'updated_by', type: 'varchar', length: 50, nullable: true })
updatedBy?: string;

@Column({ name: 'updated_at', type: 'timestampz', nullable: true })
updatedAt?: Date;

// TODO: ตรวจสอบความยาว/precision กับ DDL จริงใน sql/deploy-sbpgi-*.sql ก่อน merge
// entity ชุดนี้ไม่ประกาศ relation ตาม convention (join ด้วย raw SQL)
,
// src/entities/job-run-histories.entity.ts
import { Column, Entity, PrimaryColumn } from 'typeorm';

@Entity({ name: 'job_run_histories', schema: process.env.DB_SCHEMA })
export class JobRunHistory {
  @PrimaryColumn({ name: 'id', type: 'bigint' })
  id: number;

  @Column({ name: 'job_no', type: 'varchar', length: 5 })
  jobNo: string;

  @Column({ name: 'run_status', type: 'varchar', length: 20 })
  runStatus: string;

  @Column({ name: 'started_at', type: 'timestampz' })
  startedAt: Date;

  @Column({ name: 'finished_at', type: 'timestampz', nullable: true })
  finishedAt?: Date;

  @Column({ name: 'total_records', type: 'int', default: 0 })
  totalRecords: number;

  @Column({ name: 'success_records', type: 'int', default: 0 })
  successRecords: number;

  @Column({ name: 'failed_records', type: 'int', default: 0 })
  failedRecords: number;

  @Column({ name: 'error_message', type: 'text', nullable: true })
  errorMessage?: string;

  @Column({ name: 'triggered_by', type: 'varchar', length: 50, nullable: true })
  triggeredBy?: string;

  // TODO: ตรวจสอบความยาว/precision กับ DDL จริงใน sql/deploy-sbpgi-*.sql ก่อน merge
  // entity ชุดนี้ไม่ประกาศ relation ตาม convention (join ด้วย raw SQL)
}

```

ตารางที่เหลือของเอกสารนี้ (interface_transactions, status_email_rules) ใช้รูปแบบ entity เดียวกัน — คอลัมน์อ้างอิงจาก Database design

ตารางที่ไม่ต้องสร้าง entity เพราะใช้ของระบบเดิม/workflow engine:

Object	R/W	ใช้ของระบบเดิมตัวไหน
email_template	R/W	email_template + email_sent + @gosoft-sbp/email-lib

9.7 Repository Providers + Module wiring

```

// src/providers/sbpgi/sbpgi.ts — repository provider แบบ factory (ไม่ใช่ TypeOrmModule.forFeature)
// convention ของไฟล์เตอร์ providers คือ 1 ไฟล์ต่อโดเมน ตั้งชื่อตามโดเมน (business_user/business_user.ts,
// common_code/common_code.ts ...) ไม่ใช่ index.ts
//
// □□ ไฟล์นี้ใช้ร่วมกันทุกเอกสาร BE ของ SBPGI — ให้ **merge array เพิ่ม** เข้าไฟล์เดิม ห้ามเขียนทับ
// (ชื่อ const แยกต่อเอกสารไว้แล้วเพื่อไม่ให้ชนกัน)
import { DataSource } from 'typeorm';
import { JobConfig } from '../entities/job-configs.entity';
import { JobRunHistory } from '../entities/job-run-histories.entity';
import { InterfaceTransaction } from '../entities/interface-transactions.entity';

```

```

export const sbpgiJobBatchEmailSRMProviders = [
  {
    provide: 'JOB_CONFIG_REPOSITORY',
    useFactory: (dataSource: DataSource) => dataSource.getRepository(JobConfig),
    inject: ['DATA_SOURCE'],
  },
  {
    provide: 'JOB_RUN_HISTORIES_REPOSITORY',
    useFactory: (dataSource: DataSource) => dataSource.getRepository(JobRunHistory),
    inject: ['DATA_SOURCE'],
  },
  {
    provide: 'INTERFACE_TRANSACTION_REPOSITORY',
    useFactory: (dataSource: DataSource) => dataSource.getRepository(InterfaceTransaction),
    inject: ['DATA_SOURCE'],
  },
];

// src/modules/sbpgi-job-batch-email-srm/sbpgi-job-batch-email-srm.module.ts
import { MiddlewareConsumer, Module, NestModule } from '@nestjs/common';
import { DatabaseModule } from '../database/database.module';
import { UserContextMiddleware } from '../common/middleware/user-context.middleware';
import { WorkflowModule } from '../workflow/workflow.module';
import { sbpgiJobBatchEmailSRMProviders } from '../providers/sbpgi/sbpgi';
import { SbpgiJobBatchEmailSRMController } from './sbpgi-job-batch-email-srm.controller';
import { SbpgiJobBatchEmailSRMService } from './sbpgi-job-batch-email-srm.service';

@Module({
  imports: [DatabaseModule, WorkflowModule],
  controllers: [SbpgiJobBatchEmailSRMController],
  providers: [SbpgiJobBatchEmailSRMService, ...sbpgiJobBatchEmailSRMProviders],
  exports: [SbpgiJobBatchEmailSRMService],
})
export class SbpgiJobBatchEmailSRModule implements NestModule {
  configure(consumer: MiddlewareConsumer) {
    // ถ้าไม่ apply ตรงนี้ userId จะเป็น undefined เจียน ๆ ทุก endpoint
    consumer.apply(UserContextMiddleware).forRoutes(SbpgiJobBatchEmailSRMController);
  }
}
// TODO: register module นี้ใน app.module.ts (imports) พร้อมกับโมดูล SBPGI ตัวอื่น

```

9.8 BFF Proxy (module + controller + client service)

BFF ยังไม่มีพีเพอร์ประกันรายได้เลย จึงต้องสร้าง module ใหม่ + client service ใหม่ทั้งชุด และเลือก prefix แบบเดียวทั้งโมดูล (ที่นี้ใช้ /bff/sbpgi/...) เพื่อไม่ให้ปนแบบที่มี/ไม่มี /bff เหมือนโมดูลเดิม

```

// src/common/client-services/sbpgi-client.service.ts
import { Injectable, Logger, OnModuleInit } from '@nestjs/common';
import { BaseClientService } from './base-client.service';

@Injectable()
export class SbpgiClientService extends BaseClientService implements OnModuleInit {
  protected logger: Logger = new Logger(SbpgiClientService.name);

  onModuleInit() {
    // TODO: ถ้า deploy SBPGI แยก service ให้เพิ่ม API_SBPGI_BACKEND_* ใน AppConfigService
    // ตอนนี้ใช้ store backend ตัวเดียวกับ StoreClientService
    this.defaultHeaders[this.config.api.store.key.name] = this.config.api.store.key.value;
    this.baseUrl = this.config.api.store.url;
  }
}

// BaseClientService และ { success, data } ให้แล้ว — service ฟัง BFF จึงได้ data ตรง ๆ
// TODO: เป็น SbpgiClientService ใน providers/exports ของ ClientServiceModule (@Global)

// src/modules/sbpgi-job-batch-email-srm/sbpgi-job-batch-email-srm.service.ts (BFF)
import { Injectable } from '@nestjs/common';
import { SbpgiClientService } from '@common/client-services/sbpgi-client.service';

@Injectable()
export class SbpgiJobBatchEmailSRMBffService {
  constructor(private readonly client: SbpgiClientService) {}

  // BFF ไม่มี DB — หน้าทีเดียวก่อน user context แล้ว forward
  private userHeaders(user: any) {
    return {
      'x-user-id': user?.userId,
      'x-user-group-id': user?.groupId,
      'x-user-permissions': (user?.permissions ?? []).join(','),
    };
  }
}

```

```

getInterfacesTracking(params: any, user: any) {
  return this.client.get('/api/v1/interfaces/tracking', { params, headers: this.userHeaders(user) });
}

getInterfacesPendingAck(params: any, user: any) {
  return this.client.get('/api/v1/interfaces/pending-ack', { params, headers: this.userHeaders(user) });
}

receiveAckSta(body: any, user: any) {
  return this.client.post('/api/v1/interfaces/sta/ack', body, { headers: this.userHeaders(user) });
}
}

// ----- src/modules/sbpgi-job-batch-email-srm/sbpgi-job-batch-email-srm.controller.ts (BFF) -----
import { Body, Controller, Delete, Get, Param, Post, Put, Query, Req, UseGuards } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';

// เลือก prefix แบบเดียวทั้งโมดูล: ใช้ '/bff/sbpgi/...' (ห้ามปนกับแบบไม่มี /bff)
@Controller('bff/sbpgi/job-batch-email-srm')
@UseGuards(AuthGuard('jwt'))
export class SbpgiJobBatchEmailSRMBffController {
  constructor(private readonly service: SbpgiJobBatchEmailSRMBffService) {}

  // proxy ของ GET /api/v1/interfaces/tracking
  @Get('interfaces/tracking')
  getInterfacesTracking(@Query() query: any, @Req() req: any) {
    return this.service.getInterfacesTracking(query, req.user);
  }

  // proxy ของ GET /api/v1/interfaces/pending-ack
  @Get('interfaces/pending-ack')
  getInterfacesPendingAck(@Query() query: any, @Req() req: any) {
    return this.service.getInterfacesPendingAck(query, req.user);
  }
}

// TODO: register module ใน app.module.ts ของ BFF และเพิ่ม SbpgiClientService ใน ClientServiceModule (@Global)

```

10. Database SQL

10.1 ตารางที่อ่าน/เขียน

Table / Object	R/W	Usage
job_configs	R/W	enabled, cron, params ของ batch
job_run_histories	R/W	ประวัติการรันและสถานะล่าสุด
interface_transactions	R/W	tracking file/API interface และ ACK
status_email_rules	R	TO/CC ตามสถานะ
email_template	R/W	ใช้ของระบบเดิม: email_template + email_sent + @gosoft-sbp/email-lib

10.2 SQL จริงต่อ Endpoint

GET /api/v1/interfaces/tracking — ค้นสถานะ interface ตาม dataset/business key/status/ช่วงเวลา

```

-- □□ SQL นี้ใช้ named parameter (:name) แต่ `dataSource.query()` ของ store-backend
-- รับเฉพาะ positional $1..$n — ต้องแปลงเป็นลำดับ หรือรันผ่าน QueryBuilder
SELECT id AS tracking_id, data_name, doc_no, sent_at, return_code, acked_at AS receive_date
FROM interface_transactions
WHERE (:dataName IS NULL OR data_name = :dataName)
AND (:pending IS NULL OR return_code IS NULL)
ORDER BY sent_at DESC
LIMIT :size OFFSET :offset;

```

GET /api/v1/interfaces/pending-ack — รายการ ACK ค้างตาม watchdog rule อายุอย่างน้อย 1 วัน

```

-- □□ SQL นี้ใช้ named parameter (:name) แต่ `dataSource.query()` ของ store-backend
-- รับเฉพาะ positional $1..$n — ต้องแปลงเป็นลำดับ หรือรันผ่าน QueryBuilder
-- เกณฑ์ watchdog Job 10: return_code NULL · interface แบบไฟล์ · อายุ ≥ 1 วัน
SELECT data_name, doc_no, sent_at, (CURRENT_DATE - sent_at::date) AS age_days
FROM interface_transactions
WHERE return_code IS NULL
AND data_name IN (:staDatasets)
AND sent_at < CURRENT_DATE - 1
ORDER BY sent_at;

```

POST /api/v1/interfaces/sta/ack — STA ACK callback ให้ Job 10 เป็น safety net

```
-- □□ SQL นี้ใช้ named parameter (:name) แต่ `dataSource.query()` ของ store-backend
-- รับเฉพาะ positional $1..$n — ต้องแปลงเป็นลำดับ หรือรันผ่าน QueryBuilder
-- callback จากระบบ STA (API key) → บันทึก ACK
UPDATE interface_transactions
SET return_code = :returnCode, acked_at = :receiveDate, status = :statusAked, completed_at = :receiveDate
WHERE id = :trackingId;
```

10.3 Index / Constraint ที่ควรมี (ข้อเสนอ)

Table	DDL ที่เสนอ	ที่มา / หมายเหตุ
interface_transactions	CREATE INDEX idx_interface_transactions_pending ON interface_transactions (data_name, status, sent_at);	ข้อเสนอ — อนุมานจากเงื่อนไข query ที่เอกสารนี้ระบุ ต้องยืนยันกับ DBA ก่อนใช้จริง

ทั้งหมดเป็น ข้อเสนอ ไม่ใช่ข้อกำหนดตายตัว ข้อกำหนดทางธุรกิจ — ให้ตรวจกับ EXPLAIN ANALYZE บนข้อมูลจริง และรวมเข้าไฟล์
sql/deploy-sbpgi-*.sql แบบ idempotent (CREATE INDEX IF NOT EXISTS) ตาม pattern ที่ทีมใช้อยู่

11. Processing Flow

Step	Description
1	Receive request
2	Validate schema
3	Check idempotency
4	Process records
5	Log success/failure
6	Return summary

12. Acceptance Criteria

- job run guard prevents duplicate running job
- email preview renders variables
- failed records include detail
- ไม่มี inbound endpoint ของ SRM แล้ว (ตัด 2026-08-07) — เอกสารต้องไม่อ้างถึงอีก

13. Developer Test Checklist

No	Test
1	run job
2	run duplicate
3	interface tracking filter
4	pending ACK watchdog
5	STA ACK callback
6	email preview